Flag, Block, or Beg

A prompt asks the agent to behave. A flag records when it doesn't. A block stops it cold. They are not three flavors of one thing; they are three layers, and the measurements say which job each layer is for.

A companion to the guardrails part of a series on running Claude Code autonomously without setting your codebase on fire.

Here is a demo that lies. You notice the agent wasting turns, running `ls` to confirm a file it created one step ago, re-reading state the tool already handed back. So you add a line to `CLAUDE.md`: *trust your tool results; don't list a directory to confirm your own write.* You run it again, it behaves, and you close the ticket.

Then it runs a hundred more times while you sleep, and the line you wrote turns out to be a suggestion the agent is free to decline. The guardrails part of this series argues for hooks by design and never puts a number on the question that matters when you are choosing between them: which kind of intervention is worth reaching for, and what does each one actually buy? So here is the number. One concrete waste pattern, stopped three ways on the same task and the same model, measured. The result was not the obvious one, and it is more useful for that.

Three ways to say no

The three interventions are not three settings on one dial. They live on three different layers of Claude Code, and the layer is the whole story.

- **Beg.** A line in `CLAUDE.md` or the prompt. It shapes what the agent *tries* to do. The permissions docs put the limit in one sentence: instructions "shape what Claude tries to do, but they don't change what Claude Code allows." Persuasion, not enforcement.
- **Flag.** A `PostToolUse` hook. It fires *after* a tool call succeeds, receives the result as JSON on stdin, and can log it or hand a note back to the model. It cannot undo what already ran. Detection.
- **Block.** A `PreToolUse` hook. It fires *before* the call runs, ahead of Claude Code's own permission prompt, and returns a decision (`allow`, `deny`, `ask`, and `defer`, which hands the decision back to the normal permission flow). A `deny` means the call never happens; the agent gets your reason and chooses another move. Prevention.

```
   the agent wants to run `ls`
              |
beg    ->  advisory rule .... it may run
flag   ->  PostToolUse ...... it ran, now logged
block  ->  PreToolUse deny .. it never runs
              |
only block sits between intent and action.
```

That is the claim the design part asserts and this part measures: a rule asks, a flag records, only a block prevents. All three turned out to be true. What the design part cannot tell you is the price on the third one.

The experiment

One task, run four ways on Claude's cheap fast model, each in an isolated headless run loading *only* its own workspace policy. The task builds a tiny Python utility and, deliberately, asks the agent to run `ls` to confirm each file after writing it. That standardizes the urge, so every arm's agent has the same reason to list and the arms differ only in what the policy does about it. The four arms:

- **control** — no intervention, the baseline rate.
- **beg** — a `CLAUDE.md` rule against the redundant `ls`.
- **flag** — a `PostToolUse` hook that logs every orientation `ls`.
- **block** — a `PreToolUse` hook that denies it, with a reason attached.

Everything below is read from the CLI's own JSON and the workspace audit log. Five runs an arm; point estimates, not distributions. Crucially, each run is also scored for whether it actually *finished the task*, all three files written and `greet()` correct, so a cheap-looking arm cannot win by quietly doing less. Two of the flag arm's five runs errored out and are excluded, so its rates are over the three that ran clean; every other rate is over all five.

arm	mechanism	mean cost	mean turns	orientation `ls`	flagged	blocked	finished the task
control	none	$0.0391	9.2	2.6	—	—	2/5
beg	`CLAUDE.md` rule	$0.0391	8.2	2.6	—	—	3/5
flag	`PostToolUse`	$0.0529	13.0	4.0	4.0	—	3/3
block	`PreToolUse` deny	$0.0292	6.6	0.0	—	1.2	1/5

Read it one row at a time.

Beg was ignored in every run. The rule was obeyed, meaning zero orientation `ls`, in 0 of 5 runs. The agent listed as often with the rule as the control did without it. A standing line in `CLAUDE.md` lost to an in-the-moment instruction, every time.

Flag caught everything and prevented nothing. The hook flagged every completed orientation `ls` it saw, an exact match, 12 of 12 instances across the clean runs. Perfect detection. The completed-`ls` count did not drop, because a `PostToolUse` hook that writes to a log file is invisible to the model: it can record behavior, not change it. That invisibility is the honest claim, not "risk-free": the flag arm actually lost two of its five runs to errors and carried the highest turn count of any arm. A file-logging hook cannot cause that, so it reads as ordinary run-to-run noise, but it is noise worth naming rather than hiding. A flag is a camera: a flawless record of a mistake that still happened.

Block prevented it every time, and that is exactly where the surprise was. Completed orientation `ls`: zero in 5 of 5 runs. Prevention is total; a deny hook denies. But look at the last column: block finished the task in only 1 of 5 runs, the worst of a noisy field in which even the no-intervention control finished just 2 of 5. At five runs an arm that is a one-run gap from baseline, so read it as direction, not proof. The direction is consistent, though: block posted the lowest turn count of any arm, the signature of runs that stopped early. Denied the `ls` the task had explicitly told it to run, the agent more often lost the thread and ended without finishing. Block's low cost is not efficiency; it is, most likely, the sound of a run falling over. A hard *no*, on a step the agent believed it needed, can cost the task, not just the tokens.

Determinism is the point, not volume

Be careful what the flag arm proves, because it is easy to oversell. The honest version was written down once already, in the durable-agents experiment this method borrows from: there, a `PostToolUse` flag caught a *spontaneous* re-orientation habit 12 out of 12 times while a prompt rule cut it about 20% and was ignored on one run, and the total tool count did not fall either way. The win from a flag is not that the agent improved. It is that the mistake is caught, every time, in a record you can trust. Determinism, not volume. That record has since earned its keep at scale: the one-off flag hook grew into a per-lane registry — each team commits its own watched patterns beside the org-wide floor (the testing lane flags sleeps used as synchronization; the review lane flags any attempt to edit code) — and when a later fleet run pushed thirteen issues through the live

system, every failure it surfaced was diagnosed from the audit trail those flags write (`deploy/fleet-run-results.md`). The log the flag keeps is the log that debugged the fleet.

Which reframes beg rather than condemning it. In that earlier run the rule fought a habit and bought ~20%; here it fought an explicit instruction and bought nothing. That is not a contradiction. It is the rule's actual law: a prompt's power falls as the agent's reason to ignore it rises. Weak pull, some effect; a direct instruction to the contrary, none. A block does not have that property, because it is code in the harness, not a request to the model. That is the asymmetry the docs state plainly and most write-ups skip: a hook can *tighten* what the agent may do, but a prompt cannot *enforce* anything at all.

Prevention is not free, and neither is a hook you forgot

The block result is the one to sit with, because the naive reading, *block everything you don't want*, is the expensive one. A block is a hard stop, and a hard stop lands on the agent mid-plan. When the thing you blocked was pure danger the agent never needed, that is exactly right. When it was something the agent was in the middle of using, you do not pay in latency, you pay in derailed runs, the way a stubborn agent fighting a bad test burns a whole session. This is why the durable-agents system *flagged* the orientation waste and reserved outright `deny` for `rm -rf`, `sudo`, and `git push`, the acts with no legitimate use. Flag waste; block danger.

And there is a failure mode with no line in any table: the hook that quietly stops firing. A settings edit, a moved script, a matcher typo, and the guardrail is gone, announcing nothing, because a hook that no longer runs produces no error, only the silent absence of one. A deleted cron line makes no sound. So the hooks here ship with the guardrail every hook deserves and almost none have: eight offline tests that feed the real hook script a tool event and assert it still denies what it should, still ignores real work, and still fails open on malformed input. When a hook regresses, a test goes red instead of a mistake going quiet.

The takeaway

Three layers, three jobs, and only one of them is enforcement. But enforcement is a blunt instrument, so aim it. A rule is where you put a *preference*: the agent should generally not do this, and you accept it sometimes will. A flag is where you put an *audit*: you need to know every time this happens, even when you can't or shouldn't stop it, and because it only writes to a log it cannot change what the agent does, only record it. A block is where you put a *policy*: this must not happen, and a wrongly placed block can cost you the whole task, so you spend it only where the action has no business happening at all.

Practical rule: put preferences in prose, audits in a `PostToolUse` flag, and only the genuinely forbidden in a `PreToolUse` block with a one-line reason attached, then write the test that goes red when the block stops firing. Beg for what's nice; flag what you want to see; block what must never happen. Never confuse the log for the lock, or the lock for a nudge.

One honest boundary: these are single-digit runs on one cheap model against one waste pattern, point estimates reproducible from the repo, not a benchmark. And the hook surface is far larger than the three events used here; the current docs list dozens of events and five hook types, which is its own deep-dive. But the layer law does not depend on the count. Persuasion is not enforcement, detection is not prevention, and prevention is not free. And there is a fourth move these three do not reach, at a different boundary: a `Stop` gate on *done* itself, which turns this article's block-derails-the-run result on its head. That is the companion *Done Is Not a Claim*.

The next thing an autonomous loop needs is to decide what *green* is allowed to mean — whether an agent's own "tests passed" can be believed at all. That is measured in the companion The Agent Grades Its Own Homework.

Disclaimer: the views expressed here are my own and do not necessarily reflect those of my employer. This is a personal project, not affiliated with or endorsed by Anthropic or Temporal.

Sources

- **Claude Code hooks: the event lifecycle, the `PreToolUse` decision contract (`hookSpecificOutput.permissionDecision`, values allow / deny / ask / defer), the stdin JSON and exit-code semantics, and `PostToolUse` firing only after a tool succeeds.** `code.claude.com/docs/en/hooks`. Checked 2026-08-06. **Vendor/canonical.**
- **Permissions: "instructions … shape what Claude tries to do, but they don't change what Claude Code allows," and that a hook can tighten but not loosen what the permission layer allows.** `code.claude.com/docs/en/permissions`. Checked 2026-08-06. **Vendor/canonical.**
- **The measured runs.** Four arms times five headless runs on `claude-haiku-4-5`, isolated to workspace policy (`--setting-sources project`), everything observed from the CLI JSON plus the audit log, each run scored for task completion. Runner, hook scripts, and the eight offline hook tests: `deploy/flag-block-beg.py`, `deploy/flag-block-beg-results.md`, `tests/test_flag_block_beg.py` in the companion repo. **Practitioner.**
- **The determinism baseline (12/12 flag versus ~20% rule; "the win is determinism, not volume") and the flag-waste / block-danger split this experiment validates.** The durable-agents write-up and its `deploy/learn-loop-results.md`. **Practitioner.**